Empirical Characterization of Communication–Computation Trade-offs in Distributed Training Strategies of LLMs on NVSwitch-Connected A100 Cloud Systems

Syed Fakhar Hussain Gillani,
Tauseef ur Rehman, Seemab Latif, Muhammad Ashraf, Tariq Saeed
National University of Sciences and Technology (NUST), Islamabad, Pakistan
sgillani.msit22seecs@seecs.edu.pk, tauseef.rehman@seecs.edu.pk,
seemab.latif@seecs.edu.pk, muhammad.ashraf@seecs.edu.pk, tariq@sines.nust.edu.pk

Abstract—Large language model (LLM) training is limited not only by GPU compute but also by memory capacity and by how well the runtime overlaps collective communication with computation. We present a controlled, decomposition-based characterization of three PyTorch-native data-parallel strategies—DDP, FSDP with SHARD_GRAD_OP (FSDP-Z2) and FSDP with FULL_SHARD (FSDP-Z3)—on one node of eight A100-SXM4-80 GB GPUs connected through NVSwitch. Six dense Qwen3 models (0.6B–32B) are trained with bf16, gradient checkpointing, sequence length 2048 and per-device batch 2 across 189 runs (157 feasible, 32 out-of-memory). Each iteration is attributed to forward, backward, optimizer, exposed-communication, straggler and residual time, and the attribution closes with negligible residual. DDP leads up to 1.7B, but only nominally, since margins sit inside step-level variation. At 4B the DDP-to-FSDP transition is a near-crossover (0.7–3.2% at 4–8 GPUs), and FSDP-Z2 wins the 4B–14B range, using 38–53% less reserved memory than DDP at 4–8 GPUs (4B–8B). At the 4B/8-GPU anchor Z2's edge coincides with optimizer-state sharding, whereas FSDP-Z3 is never the fastest strategy because it issues twice the all-gathers, consistent with its longer forward pass. At 14B and 32B the choice becomes a feasibility gate: only FSDP-Z3 trains 32B. Gradient accumulation speeds up DDP (+10.0%) and FSDP-Z2 (+5.4%) but not FSDP-Z3 (+0.4%). We distill a feasibility-first selection rule.

Index Terms—distributed training, data parallelism, FSDP, ZeRO, large language models, NVSwitch, A100, communication–computation overlap

I. Introduction

Training large language models (LLMs), which build on the Transformer [1], is as much a systems problem as a modeling problem: efficiency depends on accelerator compute, memory capacity and the communication needed to synchronize workers [2]–[4]. Data parallelism is the common starting point. Distributed Data Parallel (DDP) replicates parameters, gradients and optimizer state on every GPU and all-reduces gradients [5], [6]; Fully Sharded Data Parallel (FSDP), the PyTorch realization of ZeRO [7]–[9], shards that state to save memory at the price of extra collectives.

The practitioner's question—given a model size, a GPU budget and memory headroom, when does sharding pay off?—is hard to answer from end-to-end throughput alone, because two strategies can differ for different reasons: exposed communication, memory pressure or optimizer work. Existing comparisons report throughput [10], [11], and datacenter studies characterize LLM development at scale [12], and surveys map the wider training stack [13]–[15], but few attribute iteration time to its components on a high-bandwidth intra-node fabric.

We study this on a single NVSwitch A100 node and contribute:

- A time-decomposed dataset of 189 runs (six dense Qwen3 sizes [16], three strategies, 2/4/8 GPUs, gradient-accumulation depths 1–8), with out-of-memory (OOM) cells kept as first-class feasibility results.
- A validated measurement method: a two-pass protocol, an attribution identity whose residual stays below a 5% gate, and an overlap metric cross-checked against an independent tool.
- A crossover analysis showing that the DDP-to-FSDP transition is a near-crossover whose per-step driver differs for DDP→Z2 (optimizer sharding) and Z2→Z3 (extra all-gathers), and a gradient-accumulation analysis of why accumulation helps DDP and Z2 but not Z3.
- A feasibility-first selection rule by model size and GPU count. The dataset and harness are public.[1]

II. Background and Related Work

PyTorch's distributed stack [17] offers all three strategies studied here. DDP keeps a full replica per rank and overlaps a bucketed gradient all-reduce with the backward

979-8-3195-0247-6/26/$31.00 ©2026 IEEE

[1]https://github.com/iamFakhar/ms-thesis

pass [5], [6]. FSDP-Z2 (SHARD_GRAD_OP, ZeRO-2) shards gradients and optimizer state but keeps parameters replicated during compute, using reduce-scatter and all-gather [7], [8], [18]. FSDP-Z3 (FULL_SHARD, ZeRO-3) also shards parameters, re-gathering them for both the forward and backward pass of every micro-batch [8], [19]. With gradient accumulation, DDP and Z2 can defer the gradient collective to the last micro-batch (no_sync); Z3's parameter all-gathers cannot be deferred.

Closest to our work, Patwardhan et al. [10] compare DDP and FSDP on GPT-2 and Ovi [11] evaluates DDP, FSDP and parameter servers on GPU clusters, but neither provides an iteration-level attribution of computation and exposed communication. Wang et al. [20] study FSDP memory and bandwidth sensitivity on different hardware without DDP or ZeRO-2. Zhao et al. [8] describe FSDP at production scale, and Zhang et al. [21] show that what matters is the communication left on the critical path rather than its volume. We therefore measure exposed communication separately from raw collective time, on an NVSwitch fabric [22] where NCCL [23], [24] collectives are fast. Collective-level studies [25]–[27] show that message size and pattern matter but do not compare DDP with FSDP; Chopper [28] characterizes LLM training inefficiency with GPU-level tooling, and Lagom [29] tunes communication parameters to improve overlap, whereas we use overlap as an explanatory variable for strategy choice.

III. Methodology

Platform and workload. All runs use one node with 8×A100-SXM4-80 GB [30], [31] on NVSwitch, PyTorch 2.11 [32] and NCCL 2.28.9 [33]. Measured all-reduce bus bandwidth saturates near 229 GB s^{-1} (nccl-tests [34], 8 GPUs) against a nominal 600 GB s^{-1}, reflecting a virtualized cloud node. The workload is dense Qwen3 at 0.6B, 1.7B, 4B, 8B, 14B and 32B [16], with bf16, gradient checkpointing, sequence length 2048, per-device batch 2, fused AdamW and synthetic tokens, held fixed so that global batch changes only with GPU count and accumulation. PyTorch defaults are used for bucket size and wrapping; FSDP uses BACKWARD_PRE prefetch.

Run matrix. One factorial matrix crosses model size, strategy (DDP, Z2, Z3), 2/4/8 GPUs and accumulation depth 1/2/4/8 (extension over feasible cells), plus single-GPU baselines and a checkpointing-off ablation: 189 runs, 157 feasible and 32 OOM. Each run has 60 iterations, the first 10 discarded.

Two-pass measurement. Pass 1 runs without a profiler and supplies all timing, throughput and memory (CUDA events). Pass 2 re-runs under the PyTorch profiler and supplies only NCCL device-kernel time, counts and overlap, so profiler overhead never enters the reported timings.

TABLE I
Instrument validation: claim and evidence.

Claim	Evidence
Attribution is complete	Residual ≈ 0% vs. 5% gate
Overlap is method-independent	Custom vs. HTA: +1.09, +0.96, +1.03 pp (DDP, Z2, Z3; 4B/8 GPUs)
Parser matches algorithms	Count fingerprints (Table II)
Recompute is calibrated	$r = 0.753$ (0.746, 0.760), first-order
Timing is profiler-free	Pass 1 timing only; Pass 2 for traces

TABLE II
NCCL collective counts, Qwen3-4B, 8 GPUs, GA=1

Strategy	All-reduce	All-gather	Reduce-scatter
DDP	1448	0	0
FSDP-Z2	8	296	296
FSDP-Z3	8	584	296

Attribution. The iteration is decomposed as

$$T_{\text{iter}} = T_{\text{fwd}} + \underbrace{T_{\text{bcmp}} + T_{\text{rec}} + T_{\text{comm}}^{\text{exp}}}_{T_{\text{bwd}}} + T_{\text{opt}} + T_{\text{str}} + T_{\text{res}}, \tag{1}$$

where $T_{\text{comm}}^{\text{exp}}$ is the collective time not overlapped with compute kernels in the Pass-2 timeline, $T_{\text{str}} = (R-1)T_{\text{iter}}$ uses the cross-rank max/mean ratio R, and the recompute term $T_{\text{rec}} = r\,T_{\text{fwd}}$ is calibrated with $r = 0.753$ from checkpointing-on/off runs of the two smallest models. Overlap is $\rho = |\mathcal{C}_{\text{comm}} \cap \mathcal{C}_{\text{cmp}}|/|\mathcal{C}_{\text{comm}}|$.

Validation. The residual is ≈ 0% in every run, far inside the 5% gate. On the 4B/8-GPU traces our overlap metric differs from Holistic Trace Analysis [35] by only 1.09, 0.96 and 1.03 percentage points (pp) for DDP, Z2 and Z3. Collective counts match each algorithm's fingerprint (Table II). Byte counts from NCCL device kernels are unreliable, so volume is analytical and we do not use it to explain results.

IV. Results

A. Feasibility and Throughput

Feasibility is a result, not a footnote. All strategies run through 8B. At 14B DDP is OOM at every GPU count and Z2 fails at 2 GPUs (peak 78.3 GB); Z3 clears 14B/2 GPUs at 73.3 GB. At 32B only Z3 runs, and only on 4 or 8 GPUs (Table III).

Table III shows three regimes. Up to 1.7B, DDP is nominally fastest at every GPU count (at 0.6B/8 GPUs +8.5% over Z2 and +11.4% over Z3), but with CV of 6–16% in this latency-bound regime these are not statistically separated rankings. At 4B the transition is a near-crossover. DDP leads at 2 GPUs (+2.4%), Z2 at 4 GPUs (+0.7%, a statistical tie) and at 8 GPUs (+3.2%, at the edge of the 2–3% CV bands). From 8B, Z2 leads at every GPU count: +0.2% (a tie), +2.7% and +4.0% over DDP at 2, 4 and 8 GPUs, clearly separated at 4–8 GPUs

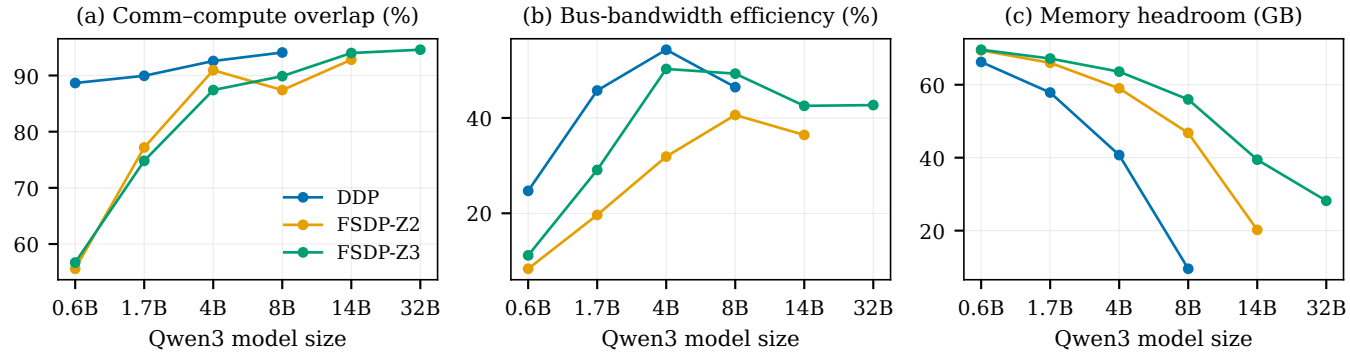


Fig. 1. Mechanism indicators versus model size at 8 GPUs, GA=1: (a) trace-based overlap, (b) analytical-volume bus bandwidth as a fraction of the nccl-tests curve, (c) headroom to 80 GB. Curves stop where a strategy is OOM.

TABLE III
Throughput (tokens/s), GA=1; bold = fastest feasible strategy. Step-level CV (single run per cell) is 6–16% for 0.6B and 1.7B, 1.4–7.6% for 4B, and ≤4.8% for 8B and above (Z3 at 8B–14B is the noisiest, 2.0–4.8%), so small-model and 4B rankings are nominal.

Model	2 GPUs			4 GPUs			8 GPUs		
	DDP	FSDP-Z2	FSDP-Z3	DDP	FSDP-Z2	FSDP-Z3	DDP	FSDP-Z2	FSDP-Z3
Qwen3-0.6B	28,148	25,569	25,243	54,206	50,341	50,435	110,630	101,999	99,290
Qwen3-1.7B	16,768	15,815	15,947	33,027	31,333	32,092	65,216	63,390	61,631
Qwen3-4B	8,421	8,220	8,074	16,389	16,500	16,137	32,341	33,385	32,351
Qwen3-8B	5,318	5,326	5,177	10,376	10,657	10,273	20,543	21,371	20,750
Qwen3-14B	OOM	OOM	3,151	OOM	6,444	6,271	OOM	12,985	12,501
Qwen3-32B	OOM	OOM	OOM	OOM	OOM	2,109	OOM	OOM	5,702

(CV ≤1.9%), and +2.8%/+3.9% over Z3 at 14B on 4/8 GPUs. Z3 is never the fastest strategy when another one fits (it edges Z2 only in a few small-model cells, inside the noise); it wins only where it is the single runnable strategy (14B/2 GPUs, 32B).

B. Iteration Budget

Backward dominates the iteration (Fig. 2); at 4B/8 GPUs it is 70–72% (Fig. 3): DDP 1013.6 ms (CV 2.1%), Z2 982.2 ms (2.8%) and Z3 1019.8 ms (9.1%). The optimizer share falls from 5.9% under DDP to 1.3% under both sharded strategies, and straggler ratios are 1.002–1.005, so cross-rank imbalance is not a first-order term on NVSwitch. Trace-based exposed communication is small but ordered: 15.7, 21.5 and 26.3 ms (1.5, 2.2, 2.6%) for DDP, Z2 and Z3. Checkpoint recomputation costs 0.75× the forward pass, not 1×.

C. Mechanism of the Crossover

The two transitions have different drivers. DDP→Z2 saves 31 ms per iteration at 4B/8 GPUs. The largest single contribution is the optimizer, which drops from 59.4 to 12.3 ms (−47 ms) because state is sharded, while the forward pass grows by 35 ms and exposed communication by 6 ms. Communication granularity is thus a mild headwind for Z2 here, not the source of its win. Z2→Z3 loses 3.1% (33,385 to 32,351 tokens/s): Z3 issues 584 all-gathers against 296 because parameters are re-gathered in forward and backward, the forward grows by a further 29 ms and exposed communication by 5 ms.

Overlap is a secondary factor from 4B up (Fig. 1a): at 4B/8 GPUs it is 93, 91 and 87% for DDP, Z2 and Z3, and Z3's overlap deficit versus Z2 closes from 8B (Z3 is about 90% at 8B and 94% at 14B–32B). Low overlap is a small-model phenomenon—Z3 climbs from 57% at 0.6B to 75, 87 and 90% at 1.7B, 4B and 8B—where per-step variance is high. Z2 also sits lowest on the nccl-tests bandwidth curve (Fig. 1b; 8–32% bus-bandwidth efficiency at 0.6B–4B against 25–54% for DDP), whereas Z3 is close to DDP from 4B (50% vs. 54%), so bandwidth efficiency alone does not explain Z3's deficit. Memory is monotone in sharding depth (Fig. 1c): at 8 GPUs reserved memory is 39 GB, 21 GB and 16 GB (4B) and 70 GB, 33 GB and 24 GB (8B) for DDP, Z2 and Z3, so at 4–8 GPUs Z2 saves 38% (4B, 4 GPUs) to 53% (8B, 8 GPUs) against DDP (about 20% at 2 GPUs).

D. Gradient Accumulation

Fig. 4 sweeps accumulation depth at 4B/8 GPUs. From depth 1 to 8, DDP rises +10.0% (32,341 to 35,564 tokens/s, CV ≤2.0%), Z2 +5.4% (33,385 to 35,188) and Z3 only +0.4% (32,351 to 32,481). The cause is visible in the collective counts: Z3 all-gathers scale with depth (584, 1168, 2336, 4672) while reduce-scatters stay at 296, because parameters must be re-gathered for every micro-batch whereas the gradient collective is issued once at

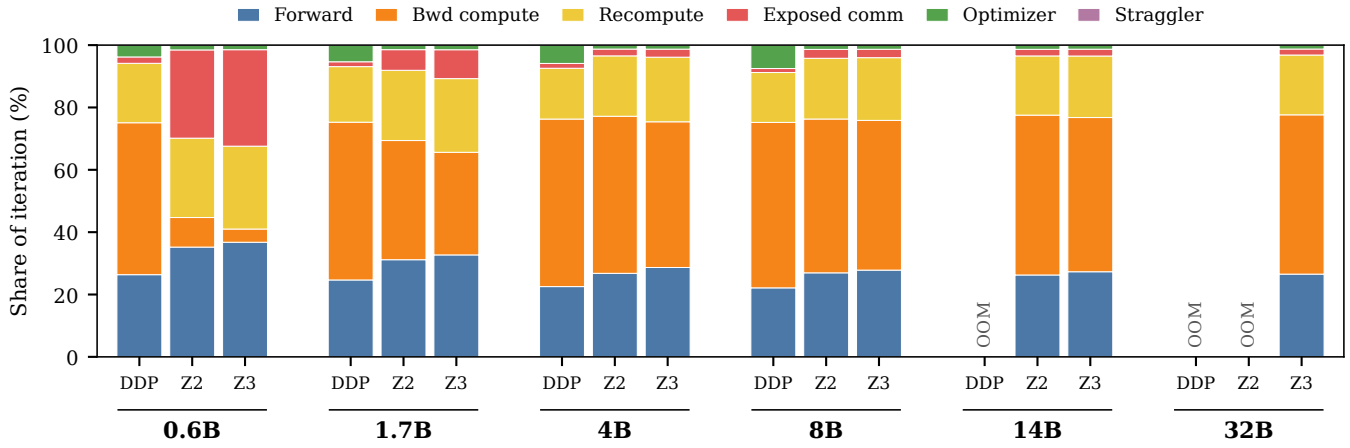


Fig. 2. Share of iteration time by attributed component at 8 GPUs, GA=1 (D = DDP, Z2/Z3 = FSDP-Z2/Z3); legend as in Fig. 3. Backward (compute, recompute, exposed communication) dominates throughout; the small-model bars show the larger communication share of the latency-bound regime.

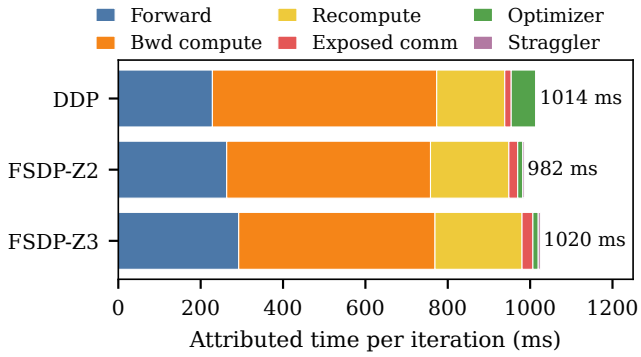


Fig. 3. Attributed iteration time, Qwen3-4B, 8 GPUs, GA=1. Residual is 0 and not drawn.

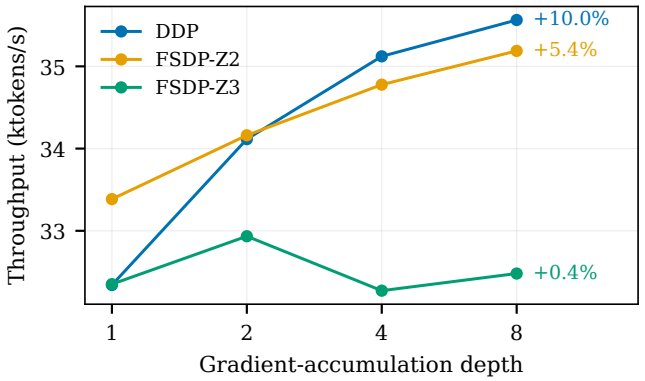


Fig. 4. Throughput versus accumulation depth, Qwen3-4B, 8 GPUs; labels give the change from depth 1 to 8.

the boundary. Accumulation therefore also moves the 4B crossover: the Z2/DDP throughput ratio falls from 1.032 to 1.001, 0.990 and 0.989, so DDP re-leads by 1.1% at depth 8 (CV 0.3–0.4%).

E. Selection Guidelines

Table IV applies the feasibility gate first and then picks the fastest feasible strategy: DDP up to 1.7B; Z2 across 4B–14B wherever it is feasible; Z3 only when memory forces it (14B/2 GPUs and 32B), where more GPUs are preferable. With accumulation, DDP and Z2 are favoured and Z3 should not be expected to benefit. The rule is empirical and tied to this node and workload.

TABLE IV
Recommended strategy (feasibility first, then throughput). Z2/Z3 = FSDP-Z2/Z3. †Near-crossover: DDP and Z2 within step-level variation.

Model	2 GPUs	4 GPUs	8 GPUs
0.6B, 1.7B	DDP	DDP	DDP
4B	DDP†	Z2†	Z2†
8B	Z2	Z2	Z2
14B	Z3 (gate)	Z2	Z2
32B	infeasible	Z3 (gate)	Z3 (gate)

V. Limitations and Future Work

Results cover one NVSwitch node of A100-80 GB GPUs (a virtualized cloud instance, so absolute bandwidth is a caveat), dense Qwen3 models, one fixed workload (batch 2, sequence 2048, chosen to isolate communication rather than maximize utilization), synthetic data and PyTorch default tuning. The recompute ratio is calibrated on two models and applied to all, overlap is cross-validated at one anchor (4B/8 GPUs), and communication volume is analytical. Convergence, multi-node fabrics, tensor/pipeline/expert parallelism and DeepSpeed are out of scope and are the natural extensions.

VI. Conclusion

On an 8×A100 NVSwitch node, DDP is nominally fastest to 1.7B, the 4B row is a near-crossover, FSDP-Z2 owns 4B–14B with up to about 50% less memory than DDP, and FSDP-Z3 is a feasibility mechanism—the only way to train 32B—that pays for itself in extra all-gathers. Gradient accumulation amortizes only deferrable collectives, helping DDP and Z2 but not Z3. The attribution is validated and the dataset and harness are released, so the choice among strategies near the boundary can be driven by mechanism rather than by a small speed gap.

References

[1] A. Vaswani et al., "Attention is all you need," in Proc. Adv. Neural Inf. Process. Syst. (NIPS), 2017.

[2] J. Duan et al., "Efficient training of large language models on distributed infrastructures: A survey," arXiv:2407.20018, 2024.

[3] K. Nagrecha, "Systems for parallel and distributed large-model deep learning training," arXiv:2301.02691, 2023.

[4] S. Ouyang, D. Dong, Y. Xu, and L. Xiao, "Communication optimization strategies for distributed deep neural network training: A survey," J. Parallel Distrib. Comput., vol. 149, pp. 52–65, 2021.

[5] S. Li et al., "PyTorch Distributed: Experiences on accelerating data parallel training," Proc. VLDB Endow., vol. 13, no. 12, pp. 3005–3018, 2020.

[6] PyTorch, "DistributedDataParallel," PyTorch Documentation, 2025. [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html. Accessed: Oct. 7, 2026.

[7] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," in Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC), 2020.

[8] Y. Zhao et al., "PyTorch FSDP: Experiences on scaling fully sharded data parallel," arXiv:2304.11277, 2023.

[9] PyTorch, "FullyShardedDataParallel," PyTorch Documentation, 2025. [Online]. Available: https://docs.pytorch.org/docs/stable/fsdp.html. Accessed: Oct. 7, 2026.

[10] I. Patwardhan, S. Gandhi, O. Khare, A. Joshi, and S. Sawant, "A comparative analysis of distributed training strategies for GPT-2," arXiv:2405.15628, 2024.

[11] M. S. I. Ovi, "Performance characterization of distributed deep learning strategies: A quantitative evaluation of DDP, FSDP, and parameter server architectures on GPU clusters," arXiv:2505.12832, 2025.

[12] Q. Hu et al., "Characterization of large language model development in the datacenter," in Proc. USENIX Symp. Netw. Syst. Design Implement. (NSDI), 2024, pp. 709–729.

[13] R. Mayer and H.-A. Jacobsen, "Scalable deep learning on distributed infrastructures: Challenges, techniques, and tools," ACM Comput. Surv., vol. 53, no. 1, pp. 1–37, 2020.

[14] X. He, F. Xue, X. Ren, and Y. You, "Large-scale deep learning optimizations: A comprehensive survey," arXiv:2111.00856, 2021.

[15] Y. Liu et al., "Understanding LLMs: A comprehensive overview from training to inference," arXiv:2401.02038, 2024.

[16] A. Yang et al., "Qwen3 technical report," arXiv:2505.09388, 2025.

[17] PyTorch, "PyTorch distributed overview," PyTorch Tutorials, 2025. [Online]. Available: https://docs.pytorch.org/tutorials/beginner/dist_overview.html. Accessed: Oct. 7, 2026.

[18] Microsoft Research, "ZeRO-2 & DeepSpeed: Shattering barriers of deep learning speed & scale," Microsoft Research Blog, 2020. [Online]. Available: https://www.microsoft.com/en-us/research/blog/zero-2-deepspeed-shattering-barriers-of-deep-learning-speed-scale/. Accessed: Oct. 7, 2026.

[19] Meta AI, "Fully sharded data parallel: Faster AI training with fewer GPUs," Meta Engineering Blog, 2021. [Online]. Available: https://engineering.fb.com/2021/07/15/open-source/fsdp/. Accessed: Oct. 7, 2026.

[20] J. Wang, J. Ebert, O. Filatov, and S. Kesselheim, "Memory and bandwidth are all you need for fully sharded data parallel," arXiv:2504.03655, 2025.

[21] Z. Zhang et al., "Is network the bottleneck of distributed training?" in Proc. ACM SIGCOMM Workshop Netw. Meets AI & ML (NetAI), 2020, pp. 8–13.

[22] NVIDIA Corp., "NVLink and NVSwitch," 2025. [Online]. Available: https://www.nvidia.com/en-us/data-center/nvlink/. Accessed: Oct. 7, 2026.

[23] Z. Hu et al., "Demystifying NCCL: An in-depth analysis of GPU communication protocols and algorithms," arXiv:2507.04786, 2025.

[24] NVIDIA Corp., "NVIDIA Collective Communications Library (NCCL) user guide," 2025. [Online]. Available: https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/. Accessed: Oct. 7, 2026.

[25] S. Lee and J. Lee, "Collective communication performance evaluation for distributed deep learning training," Appl. Sci., vol. 14, no. 12, Art. no. 5100, 2024, doi: 10.3390/app14125100.

[26] D. De Sensi et al., "Exploring GPU-to-GPU communication: Insights into supercomputer interconnects," in Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC), 2024, doi: 10.1109/SC41406.2024.00039.

[27] S. Jeaugey, "NCCL: Optimized primitives for collective multi-GPU communication," presented at OLCF Summit User Meeting, 2019. [Online]. Available: https://www.olcf.ornl.gov/wp-content/uploads/2019/12/Summit-NCCL.pdf. Accessed: Oct. 7, 2026.

[28] M. Kurzynski, S. Aga, and D. Wu, "Chopper: A multi-level GPU characterization tool and derived insights into LLM training inefficiency," arXiv:2512.08242, 2025.

[29] G. Xu et al., "Lagom: Unleashing the power of communication and computation overlapping for distributed LLM training," arXiv:2602.20656, 2026.

[30] NVIDIA Corp., "NVIDIA A100 Tensor Core GPU architecture," Whitepaper, 2020.

[31] NVIDIA Corp., "NVIDIA A100 Tensor Core GPU," 2025. [Online]. Available: https://www.nvidia.com/en-us/data-center/a100/. Accessed: Oct. 7, 2026.

[32] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2019.

[33] NVIDIA Corp., "NVIDIA Collective Communications Library (NCCL)," 2025. [Online]. Available: https://developer.nvidia.com/nccl. Accessed: Oct. 7, 2026.

[34] NVIDIA Corp., "NCCL Tests," GitHub repository, 2025. [Online]. Available: https://github.com/NVIDIA/nccl-tests. Accessed: Oct. 7, 2026.

[35] Meta, "Holistic Trace Analysis," GitHub repository, 2025. [Online]. Available: https://github.com/facebookresearch/HolisticTraceAnalysis. Accessed: Oct. 7, 2026.